

1. Ползване на API-а

В тази секция се описва как потребителите взаимодействат със системата.

- **Автентикация:** Използва се **JWT (JSON Web Token)** през протокола HTTP Bearer. Потребителят получава токен след успешен вход (/api/v1/auth/login).
- **Основни Ендпоинти:**
 - POST /api/v1/auth/register/doctor – Регистрация на лекар.
 - POST /api/v1/auth/register/patient – Регистрация на пациент.
 - POST /api/v1/appointments/ – Създаване на посещение.
 - POST /api/v1/working-hours/ – Задаване на базово работно време.
- **Грешки и кодове:**
 - 400 Bad Request: Невалидни данни, застъпване на часове или нарушаване на правилото за 24 часа.
 - 401 Unauthorized: Невалиден или липсващ токен.
 - 403 Forbidden: Опит за достъп до ресурс на друг потребител (напр. пациент се опитва да променя работно време).
 - 404 Not Found: Ресурсът не съществува.

2. Архитектура на проекта

Тук се обяснява логическата организация на кода.

- **Структура (Layered Architecture):**
 - **Models:** Дефинират таблиците в SQLite базата данни чрез SQLAlchemy (напр. Appointment, Doctor, WorkingHours).
 - **Schemas:** Pydantic модели за валидация на входните данни и форматиране на изхода (напр. UserOut за скриване на пароли).
 - **Services:** Съдържат бизнес логиката – тук се проверяват правилата за записване и работно време.
 - **Routers:** Определят URL пътищата и свързват HTTP заявките със съответните услуги.
- **Методи за проверка на време:** Използва се специализиран модул utils/date.py, който изчислява дали даден час попада в базовото работно време, постоянна промяна или временна почивка.

3. Самоанализ на работата

Оценка на качеството на кода спрямо принципите на софтуерното инженерство.

- **Спазени принципи (SOLID):**

- **Single Responsibility:** Всеки модул има една задача – моделите само за данни, услугите само за логика.
- **Interface Segregation:** Използват се различни Pydantic модели за регистрация (Register) и за изход (UserOut), за да не се излагат чувствителни данни.

- **Неспазени принципи и слабости:**

- **Fragile Checks:** Проверката на ролята чрез `user.__class__.__name__` в услугите е "stringly-typed" и лесна за счупване при рефакторинг.
- **DRY (Don't Repeat Yourself):** Проверката за съществуващ имейл се повтаря в няколко функции. Може да се обедини в една помощна функция.
- **Обработка на грешки:** Много услуги използват твърде общи `try...except` блокове или връщат стандартен код 400.
- **Конкурентност (Race Conditions):** Възможно е двама пациенти да се опитат да запишат един и същ час в една и съща милисекунда.
- **Симулиране на времето:** Тъй като логиката зависи от „текущото време“, тестовете могат да бъдат непредсказуеми.